Optimisation d'un système de préparation de livraisons

Rapport de TP – langage : Python 3 (bibliothèque standard uniquement)

1. Langage et organisation

Python a été retenu pour sa lisibilité (code proche du pseudo-code) et parce que ses structures natives – liste dynamique et table de hachage `dict` – sont exactement celles dont le projet a besoin. Le coût est une exécution plus lente qu'en C (facteur constant), sans effet sur les complexités. Le code est découpé en modules : `colis`, `tri` (tri fusion écrit à la main), `gestion` (collection), `camion` (sac à dos), `priorite`, `donnees`, `benchmark` et `main` (menu).

2. Choix algorithmiques principaux

Choix 1 – Stockage et recherche des colis

Problème	Stocker jusqu'à 100 000 colis (et plus), les ajouter et retrouver un colis par son identifiant rapidement.
Solution	Double structure : une **liste** (ordre d'insertion, parcours pour l'affichage) et une **table de hachage** `dict` id → colis. Les deux référencent les mêmes objets (pas de copie). L'ajout refuse un identifiant déjà présent.
Temps	Ajout O(1) amorti. Recherche O(1) en moyenne (O(n) dans le pire cas théorique de collisions, improbable avec des entiers).
Espace	O(n) pour la liste + O(n) pour l'index (seulement des références).
Alternative	Liste seule avec recherche linéaire O(n) ; liste triée par id + recherche dichotomique O(log n) mais insertion O(n) ; arbre binaire de recherche équilibré O(log n) partout.
Justification	La recherche est l'opération la plus fréquente : le `dict` donne le meilleur coût moyen pour un surcoût mémoire modéré. Mesure : 1 000 recherches sur 100 000 colis prennent 0,12 ms avec le dict contre ~474 ms en linéaire (×4 000).

Choix 2 – Affichage trié (poids, valeur, priorité)

Problème	Afficher les colis triés selon trois critères, sur de grandes collections.
Solution	**Tri fusion itératif** (bottom-up) écrit à la main, **stable**, avec clé calculée une seule fois par élément. Le résultat est mis en **cache** par (critère, ordre) et le cache est invalidé à chaque ajout.
Temps	O(n log n) dans tous les cas (pas de pire cas quadratique). Réaffichage sans ajout : O(1) pour obtenir la liste.
Espace	O(n) (tableau auxiliaire) + O(n) par vue en cache (au plus 6 vues).
Alternative	Tri rapide (O(n log n) en moyenne mais O(n^2) au pire, non stable) ; `sorted()` natif (Timsort, hybride fusion/insertion, O(n log n), ~20× plus rapide car écrit en C) ; tri par dénombrement O(n + k) pour la priorité (k = 5 valeurs possibles) ; maintenir des listes déjà triées avec insertion dichotomique (insertion O(n)).
Justification	Garantie O(n log n), stabilité (utile pour enchaîner des tris), implémentation itérative sans risque de dépassement de pile à 100 000 éléments. Le cache évite de retrier si la collection n'a pas changé : on paye la mémoire pour gagner du temps.

Choix 3 – Préparation du camion (sac à dos 0/1)

Problème	Choisir un sous-ensemble de colis de poids total ≤ W maximisant la valeur totale. C'est le problème du sac à dos 0/1 (NP-difficile en général).
Solution	**Programmation dynamique** : `best[w]` = meilleure valeur pour une capacité w. Pour chaque colis (p, v), on parcourt w de W à p : `best[w] = max(best[w], best[w−p] + v)` (parcours décroissant ⇒ chaque colis pris au plus une fois). Une table de décisions d'un octet par case (`bytearray`) permet de reconstruire la liste des colis chargés. Les colis plus lourds que W sont écartés d'emblée.
Temps	O(n·W) – pseudo-polynomial : linéaire en n pour une capacité fixée.
Espace	O(W) pour les valeurs + O(n·W) octets pour les décisions (100 000 colis × 51 = ~5 Mo).
Optimalité	Oui, la solution est toujours optimale, à condition que les poids soient entiers.
Alternative	Force brute O(2^n) (impossible au-delà de ~25 colis) ; **glouton** par ratio valeur/poids O(n log n), non optimal (ex. : colis (1 kg, 2), (10 kg, 10), (10 kg, 10) avec W = 20 : le glouton obtient 12, l'optimum est 20) ; séparation et évaluation (branch & bound), optimal mais exponentiel au pire.
Justification	L'énoncé exige la meilleure valeur possible, donc un algorithme exact. Avec des capacités de camion réalistes (quelques dizaines à centaines de kg) n·W reste petit. Si n·W dépasse $2 \cdot 10^7$ cases, le programme bascule automatiquement sur le glouton (complété par « meilleur colis seul », garantie ≥ 50 % de l'optimum) et l'indique à l'utilisateur.

Choix 4 – Prise en compte de la priorité

Problème	Favoriser les colis prioritaires (priorité de 1 = faible à 5 = très urgente) sans ignorer valeur et poids.
Solution	On remplace la valeur par une **utilité** : `utilité = valeur × (1 + α·(priorité − 1) / 4)`, avec α = 1 par défaut (priorité 5 ⇒ valeur doublée), puis on réutilise la même programmation dynamique. Le poids reste traité par la contrainte de capacité.
Temps / Espace	Inchangés : O(n·W) / O(n·W). Le calcul de l'utilité est O(1) par colis.
Alternative	Charger d'abord tous les colis de priorité maximale puis compléter (règle stricte, mais peut gaspiller de la capacité) ; bonus additif `valeur + β·priorité` (dépend de l'échelle des valeurs) ; optimisation lexicographique (priorité d'abord, valeur ensuite).
Justification	Bonus multiplicatif : proportionnel à la valeur, indépendant de l'unité, réglable (α = 0 ⇒ problème initial, α grand ⇒ priorité dominante). La sélection reste optimale pour le critère choisi. Exemple de l'énoncé (W = 15) : même chargement 101+104+105, valeur 60, utilité 91,25. Autre exemple : un colis (10 kg, 30 €, prio 1) est remplacé par un colis (10 kg, 25 €, prio 5).

3. Recherche et performances

Mesures (`python3 src/benchmark.py`, Mac, Python 3.13) – temps en millisecondes, 1 000 recherches, capacité 50 kg, utilité avec priorité :

n	ajout total	1000 rech. dict	1000 rech. linéaires	tri fusion	sorted()	sac à dos DP	glouton
100	0,01	0,05	0,9	0,07	0,01	0,1	0,10
1 000	0,09	0,05	6,5	1,00	0,08	1,2	1,11
10 000	0,63	0,05	53,4	10,8	0,68	10,4	12,4
100 000	5,96	0,12	474,3	123,7	6,30	102,2	161,7

Influence de la capacité (n = 1 000) : DP 1,1 ms (W = 50), 14 ms (W = 500), 176 ms (W = 5 000) ; glouton ≈ 1 ms quel que soit W. Sur ces données aléatoires le glouton est à moins de 0,01 % de l'optimum, mais il peut être bien pire sur des cas défavorables.

Opération 1 – Recherche par identifiant

O(1) moyen avec le dict : le temps reste constant de 100 à 100 000 colis. La version linéaire O(n) est multipliée par ~10 quand n est multiplié par 10. Le choix reste adapté ; aucune amélioration nécessaire, sauf si la mémoire devenait critique (on pourrait alors utiliser un tableau trié + dichotomie).

Opération 2 – Tri

O(n log n) : ×10 sur n donne un peu plus que ×10 sur le temps (10,8 → 123,7 ms). Adapté jusqu'à plusieurs millions d'éléments. Amélioration possible : utiliser `sorted()` (même complexité, constante bien plus faible) ou un tri par dénombrement O(n) pour la priorité, qui n'a que 5 valeurs.

Opération 3 – Sac à dos

O(n·W) : linéaire en n à W fixé (×10 sur n ⇒ ×10 sur le temps), mais aussi linéaire en W, et la table de décisions occupe n·W octets. C'est l'opération la plus risquée. Amélioration : pré-filtrer les colis (ex. ne garder que les meilleurs ratios par tranche de poids, car au plus W colis de poids ≥ 1 peuvent tenir dans le camion), réduire la mémoire de reconstruction (algorithme de Hirschberg, O(W) d'espace) ou basculer sur le glouton / branch & bound.

4. Questions de synthèse

Choix le plus important. La programmation dynamique pour remplir le camion : c'est la seule méthode qui trouve toujours la meilleure valeur en un temps raisonnable. Le glouton ne garantit pas le meilleur résultat et tester toutes les combinaisons serait beaucoup trop long.

Partie problématique avec plusieurs millions de colis. La préparation du camion : elle utilise beaucoup de mémoire (nombre de colis × capacité) et devient lente. Dans ce cas, le programme passe au glouton, plus rapide mais pas toujours optimal.

Vitesse contre mémoire. J'ai privilégié la vitesse : le dictionnaire permet de retrouver un colis immédiatement mais prend plus de mémoire, et les listes triées sont gardées en mémoire pour ne pas refaire le tri à chaque affichage.

Meilleur résultat pour le camion ? Non : la programmation dynamique donne déjà la solution optimale. Une autre méthode pourrait seulement être plus rapide, pas donner une meilleure valeur.

Refaire le projet en C. Les algorithmes resteraient les mêmes. Seule l'implémentation changerait : gestion de la mémoire à la main et table de hachage à écrire soi-même. Le programme serait aussi beaucoup plus rapide.